

ASSIGNMENT NO.7: Subqueries and Nested Queries

Q1) Create a MySQL database named CollegeDB and create two tables: Course and StudentResult.

Course Table

Column Name	Data Type	Constraint
CourseID	INT	PRIMARY KEY
CourseName	VARCHAR(50)	NOT NULL
Fees	DECIMAL(10,2)	NOT NULL

StudentResult Table

Column Name	Data Type	Constraint
StudentID	INT	PRIMARY KEY
StudentName	VARCHAR(50)	NOT NULL
CourseID	INT	FOREIGN KEY
City	VARCHAR(30)	
Marks	INT	

Insert at least 4 records into Course and 10 records into StudentResult.

QUERY

```
-- Select existing database
```

```
USE CollegeDB;
```

```
-- Q1) CREATE TABLES
```

```
CREATE TABLE Course_Subquery (
```

```
    CourseID INT PRIMARY KEY,
```

```
    CourseName VARCHAR(50) NOT NULL,
```

```
    Fees DECIMAL(10,2) NOT NULL
```

```
);
```

```
-- StudentResult table for Assignment 7
```

```
CREATE TABLE StudentResult_Subquery (
```

```
    StudentID INT PRIMARY KEY,
```

```
    StudentName VARCHAR(50) NOT NULL,
```

```
    CourseID INT,
```

```
City VARCHAR(30),  
Marks INT,  
FOREIGN KEY (CourseID) REFERENCES Course_Subquery(CourseID)  
);
```

-- Insert 4 records into Course table

```
INSERT INTO Course_Subquery
```

```
(CourseID, CourseName, Fees)
```

```
VALUES
```

```
(101, 'BSc IT', 45000.00),
```

```
(102, 'BCA', 40000.00),
```

```
(103, 'BSc CS', 42000.00),
```

```
(104, 'Data Science', 55000.00);
```

-- Insert 10 records into StudentResult table

```
INSERT INTO StudentResult_Subquery
```

```
(StudentID, StudentName, CourseID, City, Marks)
```

```
VALUES
```

```
(1, 'Rahul Sharma', 101, 'Mumbai', 78),
```

```
(2, 'Priya Patil', 102, 'Pune', 85),
```

```
(3, 'Aman Verma', 103, 'Nashik', 67),
```

```
(4, 'Sneha Joshi', 101, 'Mumbai', 91),
```

```
(5, 'Karan Mehta', 102, 'Thane', 74),
```

```
(6, 'Neha Shah', 101, 'Pune', 88),
```

```
(7, 'Rohan Desai', 103, 'Mumbai', 69),
```

```
(8, 'Meera Kulkarni', 102, 'Nashik', 82),
```

```
(9, 'Arjun Nair', 101, 'Thane', 59),
```

```
(10, 'Divya More', 102, 'Mumbai', 93);
```

```
SELECT * FROM Course_Subquery;
```

```
SELECT * FROM StudentResult_Subquery;
```

phpMyAdmin

Recent Favorites

Server: 127.0.0.1 » Database: collegedb » Table: StudentResult_Subquery

Browse Structure SQL Search Insert Export Import

Show query box

Showing rows 0 - 3 (4 total, Query took 0.0016 seconds.)

SELECT * FROM Course_Subquery;

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key:

Extra options

		CourseID	CourseName	Fees
☐	Edit Copy Delete	101	BSc IT	45000.00
☐	Edit Copy Delete	102	BCA	40000.00
☐	Edit Copy Delete	103	BSc CS	42000.00
☐	Edit Copy Delete	104	Data Science	55000.00

Check all | With selected: Edit Copy Delete Export

Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key:

phpMyAdmin

Recent Favorites

Server: 127.0.0.1 » Database: collegedb » Table: StudentResult_Subquery

Browse Structure SQL Search Insert Export Import Privileges

SELECT * FROM StudentResult_Subquery;

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Extra options

		StudentID	StudentName	CourseID	City	Marks
☐	Edit Copy Delete	1	Rahul Sharma	101	Mumbai	78
☐	Edit Copy Delete	2	Priya Patil	102	Pune	85
☐	Edit Copy Delete	3	Aman Verma	103	Nashik	67
☐	Edit Copy Delete	4	Sneha Joshi	101	Mumbai	91
☐	Edit Copy Delete	5	Karan Mehta	102	Thane	74
☐	Edit Copy Delete	6	Neha Shah	101	Pune	88
☐	Edit Copy Delete	7	Rohan Desai	103	Mumbai	69
☐	Edit Copy Delete	8	Meera Kulkarni	102	Nashik	82
☐	Edit Copy Delete	9	Arjun Nair	101	Thane	59
☐	Edit Copy Delete	10	Divya More	102	Mumbai	93

Check all | With selected: Edit Copy Delete Export

Q2) Write a MySQL query to display students whose marks are greater than the average marks of all students.

The output should contain:

Column
StudentName
Marks

QUERY

```
SELECT StudentName, Marks
FROM StudentResult_Subquery
WHERE Marks > (
    SELECT AVG(Marks)
    FROM StudentResult_Subquery
);
```

The screenshot shows the phpMyAdmin interface. On the left is the database structure tree, and on the right is the SQL query editor and results pane. The query executed is:

```
SELECT StudentName, Marks FROM StudentResult_Subquery WHERE Marks > ( SELECT AVG(M
```

The results pane shows 5 rows of data:

StudentName	Marks
Priya Patil	85
Sneha Joshi	91
Neha Shah	88
Meera Kulkarni	82
Divya More	93

Q3) Write a MySQL query to display all students enrolled in the BCA course using a subquery.

The output should contain:

Column
StudentID
StudentName

Column
CourseID
Marks

QUERY

```
SELECT StudentID, StudentName, CourseID, Marks
```

```
FROM StudentResult_Subquery
```

```
WHERE CourseID = (
```

```
    SELECT CourseID
```

```
    FROM Course_Subquery
```

```
    WHERE CourseName = 'BCA'
```

```
);
```

The screenshot shows the phpMyAdmin interface with the following details:

- Database:** collegedb
- Table:** StudentResult_Subquery
- Query:** `SELECT StudentID, StudentName, CourseID, Marks FROM StudentResult_Subquery WHERE CourseName = 'BCA' ;`
- Result:** Showing rows 0 - 3 (4 total, Query took 0.0198 seconds.)
- Data Table:**

	StudentID	StudentName	CourseID	Marks
☐	2	Priya Patil	102	85
☐	5	Karan Mehta	102	74
☐	8	Meera Kulkarni	102	82
☐	10	Divya More	102	93

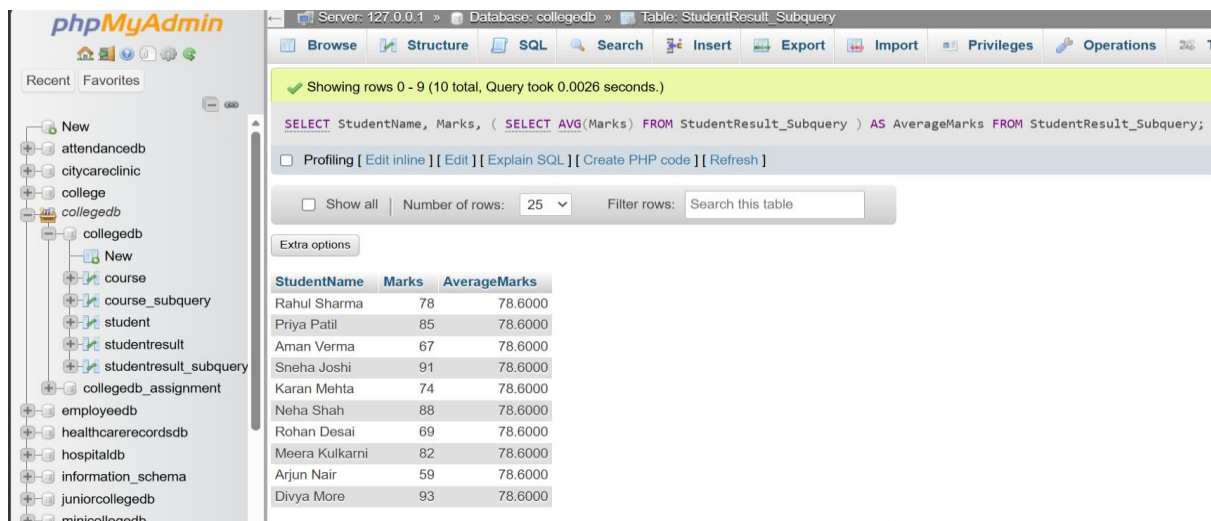
Q4) Write a MySQL query to display each student with the overall average marks using a subquery in the SELECT clause.

The output should contain:

Column
StudentName
Marks
AverageMarks

QUERY

```
SELECT StudentName, Marks,  
  
    (  
        SELECT AVG(Marks)  
        FROM StudentResult_Subquery  
    ) AS AverageMarks  
FROM StudentResult_Subquery;
```



Server: 127.0.0.1 » Database: collegedb » Table: StudentResult_Subquery

Showing rows 0 - 9 (10 total, Query took 0.0026 seconds.)

`SELECT StudentName, Marks, ( SELECT AVG(Marks) FROM StudentResult_Subquery ) AS AverageMarks FROM StudentResult_Subquery;`

☐ Profiling [[Edit inline](#)] [[Edit](#)] [[Explain SQL](#)] [[Create PHP code](#)] [[Refresh](#)]

☐ Show all | Number of rows: 25 | Filter rows:

Extra options

StudentName	Marks	AverageMarks
Rahul Sharma	78	78.6000
Priya Patil	85	78.6000
Aman Verma	67	78.6000
Sneha Joshi	91	78.6000
Karan Mehta	74	78.6000
Neha Shah	88	78.6000
Rohan Desai	69	78.6000
Meera Kulkarni	82	78.6000
Arjun Nair	59	78.6000
Divya More	93	78.6000

Q5) Write a MySQL query using a correlated subquery to display students who scored more than their own course average.

The output should contain:

Column
StudentID
StudentName
CourseID
Marks

QUERY

```
SELECT StudentID, StudentName, CourseID, Marks  
  
FROM StudentResult_Subquery S1  
  
WHERE Marks > (  
    SELECT AVG(Marks)
```

```

FROM StudentResult_Subquery S2
WHERE S2.CourseID = S1.CourseID
);

```

The screenshot shows the phpMyAdmin interface. On the left is the database navigation tree. The main panel displays a SQL query for the 'StudentResult_Subquery' table in the 'collegedb' database. The query is a correlated subquery that calculates the average marks for each student across all courses they are enrolled in.

Query:

```

SELECT StudentID, StudentName, CourseID, Marks FROM StudentResult_Subquery S1 WHERE Marks > ( SELECT AVG(Marks)
StudentResult_Subquery S2 WHERE S2.CourseID = S1.CourseID );

```

Results:

StudentID	StudentName	CourseID	Marks
2	Priya Patil	102	85
4	Sneha Joshi	101	91
6	Neha Shah	101	88
7	Rohan Desai	103	69
10	Divya More	102	93

Q6) Write MySQL queries to display courses having students and courses having no students.

Use:

Operator
EXISTS
NOT EXISTS

QUERY

```

SELECT CourseID, CourseName
FROM Course_Subquery C
WHERE EXISTS (
    SELECT 1
    FROM StudentResult_Subquery S
    WHERE S.CourseID = C.CourseID
);

```

The screenshot shows the phpMyAdmin interface. On the left is the database navigation tree with 'collegedb' selected. The main panel displays a SQL query for the 'Course_Subquery' table:

```
SELECT CourseID, CourseName FROM Course_Subquery C WHERE EXISTS ( SELECT 1 FROM StudentResult_S );
```

The query results show 3 rows:

CourseID	CourseName
101	BSc IT
102	BCA
103	BSc CS

-- Q6) COURSES HAVING NO STUDENTS

-- USING NOT EXISTS

-- =====

QUERY

```
SELECT CourseID, CourseName
FROM Course_Subquery C
WHERE NOT EXISTS (
    SELECT 1
    FROM StudentResult_Subquery S
    WHERE S.CourseID = C.CourseID
);
```

phpMyAdmin

Recent

Favorites

New

attendanceadb

citycareclinic

college

collegedb

collegedb

New

course

course_subquery

student

studentresult

studentresult_subquery

collegedb_assignment

employeedb

healthcarerecordsdb

hospitaldb

information_schema

juniorcollegedb

minicollegedb

Server: 127.0.0.1 » Database: collegedb » Table: Course_Subquery

Browse

Structure

SQL

Search

Insert

Export

Import

Pr

Showing rows 0 - 0 (1 total, Query took 0.0188 seconds.)

-- Q6) COURSES HAVING NO STUDENTS -- USING NOT EXISTS -- =====
Course_Subquery C WHERE NOT EXISTS (SELECT 1 FROM StudentResult_Subquery S WHERE S.CourseID

[Edit inline] [Edit] [Create PHP code]

Show all

 | Number of rows: 25 | Filter rows: Search this table

Extra options

← T →

CourseID

CourseName

☐

Edit

Copy

Delete

104

Data Science

↑

☐ Check all

With selected:

Edit

Copy

Delete

Export

Show all

 | Number of rows: 25 | Filter rows: Search this table

Query results operations

Print

Copy to clipboard

Export

Display chart

Create view